Design Patterns for Microservices Architecture

DAY 149
06/Dec/2025

Design Patterns for Microservices Architecture

Below are the main design pattern of microservices:

1. API Gateway Pattern

API Gateway pattern simplifies the client's experience by hiding the complexities of multiple services behind one interface. It can also handle tasks like authentication, logging, and rate limiting, making it a crucial part of microservices architecture.

2. Service Registry Pattern

Service Registery pattern is like a phone book for microservices. It maintains a list of all active services and their locations (network addresses). When a service starts, it registers itself with the registry.

Other services can then look up the registry to find and communicate with it. This dynamic discovery enables flexibility and helps services interact without hardcoding their locations.

3. Circuit Breaker Pattern

In circuit breaker pattern If a service fails repeatedly, the circuit breaker trips, preventing further requests to that service. After a timeout period, it allows limited requests to test if the service is back online. This reduces the load on failing services and enhances system resilience.

4. Saga Pattern

Saga pattern is useful for managing complex business processes that span multiple services. Instead of treating the process as a single transaction, the saga breaks it down into smaller steps, each handled by different services.

If one step fails, compensating actions are taken to reverse the previous steps. This way, you maintain data consistency across the system, even in the face of failures.

5. Event Sourcing Pattern

In Event Sourcing Pattern, Each event describes a change that occurred, allowing services to reconstruct the current state by replaying the event history. This provides a clear audit trail and simplifies data recovery in case of errors.

6. Strangler Pattern

Strangler pattern allows for a gradual transition from a monolithic application to microservices. New features are developed as microservices while the old system remains in use.

Over time, as more functionality is moved to microservices, the old system is gradually "strangled" until it can be fully retired. This approach minimizes risk and allows for a smoother migration.

7. Bulkhead Pattern

Similar to compartments in a ship, the bulkhead pattern isolates different services to prevent failures from affecting the entire system.

If one service encounters an issue, it won't compromise others. By creating boundaries, this pattern enhances the resilience of the system, ensuring that a failure in one area doesn't lead to a total system breakdown.

8. API Composition Pattern

When you need to gather data from multiple microservices, the API composition pattern helps you do so efficiently.

A separate service (the composition service) collects responses from various services and combines them into a single response for the client. This reduces the need for clients to make multiple requests and simplifies their interaction with the system.

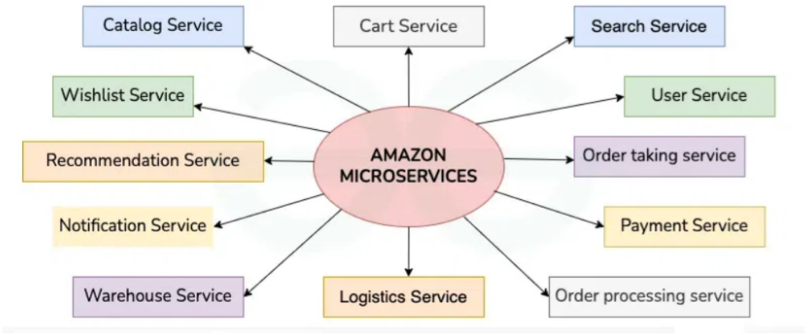
9. CQRS Design Pattern

CQRS Design Pattern divides the way data is handled into two parts: commands and queries. Commands are used to change data, like creating or updating records, while queries are used just to fetch data. This separation allows you to tailor each part for its specific purpose.

Real-World Example of Microservices

Let's understand the Miscroservices using the real-world example of Amazon E-Commerce Application:

Amazon's online store runs on many small, specialized microservices, each handling a specific task. Working together, they create a smooth shopping experience.



Below are the microservices involved in Amazon E-commerce Application:

- **User Service**: Handles user accounts and preferences, making sure each person has a personalized experience.
- **Search Service**: Helps users find products quickly by organizing and indexing product information.
- **Catalog Service**: Manages the product listings, ensuring all details are accurate and easy to access.
- **Cart Service**: Lets users add, remove, or change items in their shopping cart before checking out.
- **Wishlist Service**: Allows users to save items for later, helping them keep track of products they want.
- **Order Taking Service**: Processes customer orders, checking availability and validating details.
- **Order Processing Service**: Oversees the entire fulfillment process, working with inventory and shipping to get orders delivered.
- **Payment Service**: Manages secure transactions and keeps track of payment details.
- **Logistics Service**: Coordinates everything related to delivery, including shipping costs and tracking.
- **Warehouse Service**: Keeps an eye on inventory levels and helps with restocking when needed.
- **Notification Service**: Sends updates to users about their orders and any special offers.
- **Recommendation Service**: Suggests products to users based on their browsing and purchase history